

public interface Set<E> extends Iterable<E>

public boolean contains(E e)	Returns true if this set contains the specified element.
public void add(E e)	Adds the specified element to this set if it is not already present.
public E remove(E e)	Removes and returns the specified element from this set if it is present.
public int size()	Returns the number of elements in this set.

public interface Map<K, V>

public boolean containsKey(K key)	Returns true if this map contains a mapping for the specified key.
public V get(K key)	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
public void put(K key, V value)	Associates the specified value with the specified key in this map.
public V remove(K key)	Removes the mapping for a key from this map if it is present.
public int size()	Returns the number of key-value mappings in this map.
public Set<K> keySet()	Returns the set of keys in this map.

public interface List<E> extends Iterable<E>

public boolean contains(E element)	Returns true if this list contains the specified element.
public E get(int index)	Returns the element at the specified position in this list.
public void set(int index, E element)	Replaces the element at the specified position in this list with the specified element.
public void add(E e)	Appends the specified element to the end of this list.
public E remove(int index)	Removes and returns the element at the specified position in this list.
public int size()	Returns the number of elements in this list.
public static <E> List<E> of(E... elements)	Returns an immutable list containing an arbitrary number of elements. Example: List<Integer> lst = List.of(1, 2, 3);

Implementations of Lists, Sets, and Maps:

```
public class ArrayList<E> implements List<E> {...}
public class LinkedList<E> implements List<E> {...}
public class HashSet<E> implements Set<E> {...}
public class TreeSet<E> implements Set<E> {...}
public class HashMap<K, V> implements Map<K, V> {...}
public class TreeMap<K, V> implements Map<K, V> {...}
```

Note: Some method signatures are slightly different than in the actual Java language specification.

public class Math

public static double pow(double x, double y)	Returns the value of x raised to the power of y.
public static long floorMod(long x, long y)	Returns the positive remainder when dividing x by y.
public static double sqrt(double x)	Returns the positive square root of x.
public static int min(int x, int y)	Returns the smaller of two int values.
public static int max(int x, int y)	Returns the greater of two int values.
public static double abs(double a)	Returns the absolute value of a double value.

Integer.MIN_VALUE = -2147483648 = -2^{31}

Integer.MAX_VALUE = 2147483647 = $2^{31} - 1$

public class IntList

public int first	Instance variable. Value of the first element of the IntList.
public IntList rest	Instance variable. Reference to the rest of this IntList.
public static IntList of(int... elements)	Returns an IntList containing an arbitrary number of elements. Example: IntList lst = IntList.of(1, 2, 3);

public interface Deque61B<E> extends Iterable<E>

public E getFirst()	Retrieves, but does not remove, the first element of this Deque61B.
public E getLast()	Retrieves, but does not remove, the last element of this Deque61B.
public void addFirst(E e)	Inserts the specified element at the front of this Deque61B.
public void addLast(E e)	Inserts the specified element at the end of this Deque61B.
public E removeFirst()	Retrieves and removes the first element of this Deque61B.
public E removeLast()	Retrieves and removes the last element of this Deque61B.
public E get(int i)	Gets the ith element of this Deque61B.
public int size()	Returns the number of elements in this Deque61B.
public Iterator<E> iterator()	Returns an iterator over this Deque61B.

Implementations of Deques covered in class:

```
public class ArrayDeque61B<E> implements Deque61B<E> {...}
```

```
public class LinkedListDeque61B<E> implements Deque61B<E> {...}
```

Example Google Truth tests:

```
assertThat(x).isEqualTo(y)
```

```
assertThat(x).containsExactly("a", "b", "c").inOrder()
```

```
assertThat(x).isFalse()
```

```
assertThat(x).isNull()
```

```
assertThat(x).isEmpty()
```

```
assertThat(x).isTrue()
```